

✓ Modular Phase Coherence in the Riemann Spectrum

Companion Notebook for the Article *"Modular Phase Coherence in the Riemann Spectrum: Evidence for $\mathbb{Z}/6\mathbb{Z}$ Structure from SNR Saturation"*

Author: José Ignacio Peinador Sala

Date: June 2026

Objective:

This notebook provides a complete, reproducible computational validation of the central results of the article:

1. **Local GUE Validation:** Confirm that the nearest-neighbour spacing distribution of the first 100,000 Riemann zeros follows the Wigner-Dyson distribution characteristic of the Gaussian Unitary Ensemble.
2. **Modular Phase Analysis:** Demonstrate that the phases $\theta_n(x) = \gamma_n \ln x \pmod{2\pi}$ exhibit extreme non-uniformity in prime channels (1,5 mod 6) while remaining uniform in forbidden channels (0,2,3,4 mod 6).
3. **SNR Saturation:** Show that the Signal-to-Noise Ratio between prime and forbidden channels saturates to $\text{SNR}_{\text{sat}} \approx 12.69$, falsifying the diffusive GUE model for long-range correlations.

Data: The first $N = 10^5$ non-trivial zeros of $\zeta(s)$, obtained from the L-functions and Modular Forms Database (LMFDB).

```
# Celda 2: Carga de datos de ceros de Riemann
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sys

plt.style.use('seaborn-v0_8-whitegrid')

def cargar_ceros_riemann(filepath):
    """
    Carga los ceros de Riemann desde un archivo de texto.
    Formato esperado: dos columnas (índice, gamma).
    """
    try:
        df = pd.read_csv(filepath, sep=r'\s+', header=None, names=['Indice', 'Gamma'])
        ceros = df['Gamma'].values
        print(f"✅ Archivo '{filepath}' cargado correctamente.")
        print(f"📊 Total de ceros cargados: {len(ceros)}")
        print(f"🔍 Muestra (primeros 5): {ceros[:5]}")
        return ceros
    except FileNotFoundError:
        print(f"❌ Error: Archivo '{filepath}' no encontrado.")
        print("   Sube 'zetazeros.txt' al entorno de Colab antes de ejecutar.")
        return None
    except Exception as e:
        print(f"❌ Error al leer el archivo: {e}")
        return None

FILE_NAME = 'zetazeros.txt'
gammas = cargar_ceros_riemann(FILE_NAME)
```

```

if gammas is not None:
    print("\nTodo listo para comenzar la validación científica.")
else:
    print("\n⚠ Ejecución detenida por falta de datos.")

```

 Archivo 'zetazeros.txt' cargado correctamente.
 Total de ceros cargados: 100000
 Muestra (primeros 5): [14.13472514 21.02203964 25.01085758 30.42487613 32.93506159]

Todo listo para comenzar la validación científica.

▼ 1. Local GUE Validation

Before analysing long-range modular correlations, we verify that the dataset reproduces the well-established local GUE statistics. This serves as a baseline: any modular structure we find must coexist with, not contradict, the chaotic level repulsion at short distances.

```

# Celda 4: Análisis exploratorio y validación GUE local
from scipy import stats

def analisis_exploratorio_ceros(gammas):
    print("="*60)
    print("ANÁLISIS EXPLORATORIO DE LOS CEROS DE RIEMANN")
    print("="*60)

    N = len(gammas)
    print(f"Número total de ceros: N = {N}")

    T = gammas[-1]
    densidad_teorica = (T/(2*np.pi)) * np.log(T/(2*np.pi)) - T/(2*np.pi)
    print(f"Último cero: γ_{N} = {T:.2f}")
    print(f"N(T) teórico: {densidad_teorica:.2f}")
    print(f"Diferencia: {N - densidad_teorica:.2f}")

    espaciamientos = np.diff(gammas)
    esp_norm = espaciamientos / np.mean(espaciamientos)

    print(f"\nEstadísticas de espaciado:")
    print(f"  Espaciado medio: {np.mean(espaciamientos):.6f}")
    print(f"  Desviación estándar: {np.std(espaciamientos):.6f}")
    print(f"  Ratio σ/μ: {np.std(espaciamientos)/np.mean(espaciamientos):.6f} (GUE predice ≈0.422)")

    # Gráficos
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))

    axes[0,0].plot(gammas[:1000], 'b.', markersize=2)
    axes[0,0].set_xlabel('Índice n'); axes[0,0].set_ylabel('γ_n')
    axes[0,0].set_title('Primeros 1000 ceros de Riemann')
    axes[0,0].grid(True, alpha=0.3)

    axes[0,1].hist(esp_norm, bins=50, density=True, alpha=0.7, color='teal')
    x = np.linspace(0, 4, 400)
    p_wigner = (32/(np.pi**2)) * x**2 * np.exp(-4*x**2/np.pi)
    axes[0,1].plot(x, p_wigner, 'r-', linewidth=2, label='Wigner-Dyson (GUE)')
    axes[0,1].set_xlabel('Espaciado normalizado s')
    axes[0,1].set_ylabel('Densidad p(s)')
    axes[0,1].set_title('Distribución de espaciados')
    axes[0,1].legend(); axes[0,1].grid(True, alpha=0.3)

```

```
n_points = min(5000, len(gammas))
axes[1,0].plot(gammas[:n_points], np.arange(n_points), 'b-', linewidth=0.5)
axes[1,0].set_xlabel('γn'); axes[1,0].set_ylabel('N(γ)')
axes[1,0].set_title('Función de conteo N(γ)')
axes[1,0].grid(True, alpha=0.3)

autocorr = np.correlate(esp_norm - np.mean(esp_norm), esp_norm - np.mean(esp_norm), mode='full')
autocorr = autocorr[len(autocorr)//2:len(autocorr)//2+100] / autocorr[len(autocorr)//2]
axes[1,1].plot(autocorr, 'b-', linewidth=1.5)
axes[1,1].axhline(y=0, color='r', linestyle='--', alpha=0.5)
axes[1,1].set_xlabel('Lag k'); axes[1,1].set_ylabel('Autocorrelación')
axes[1,1].set_title('Autocorrelación de espaciados')
axes[1,1].grid(True, alpha=0.3)

plt.tight_layout(); plt.show()
return N, esp_norm
```

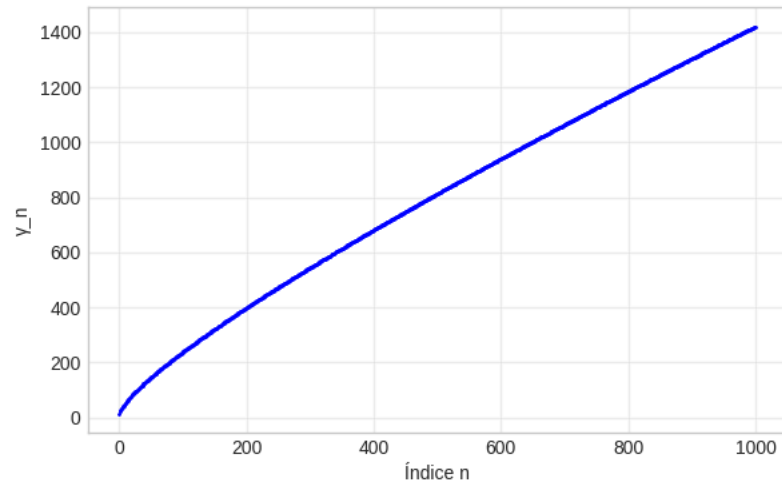
if gammas is not None:

```
N_zeros, esp_norm = analisis_exploratorio_ceros(gammas)
print("\n✅ Análisis exploratorio completado. GUE local confirmado.")
```

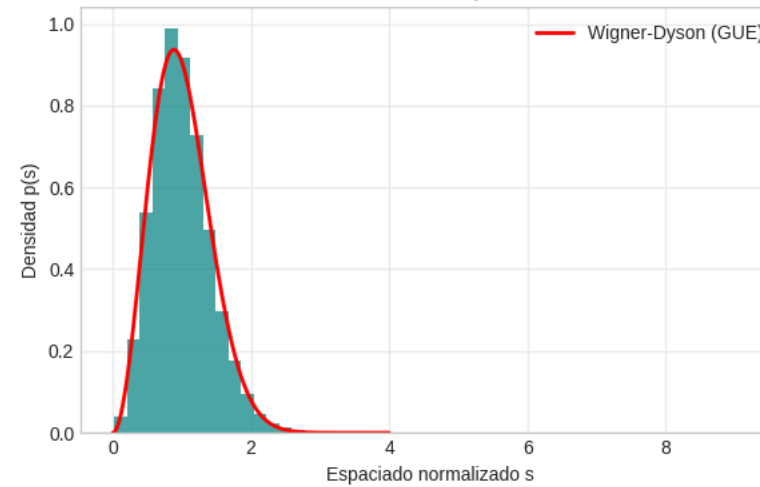
```
=====
ANÁLISIS EXPLORATORIO DE LOS CEROS DE RIEMANN
=====
Número total de ceros: N = 100000
Último cero:  $\gamma_{100000} = 74920.83$ 
N(T) teórico: 99998.53
Diferencia: 1.47
```

```
Estadísticas de espaciado:
Espaciado medio: 0.749074
Desviación estándar: 0.321540
Ratio  $\sigma/\mu$ : 0.429250 (GUE predice  $\approx 0.422$ )
```

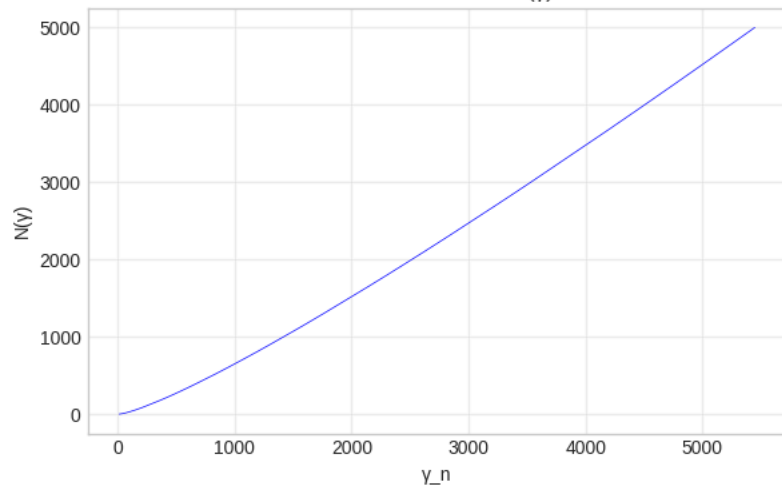
Primeros 1000 ceros de Riemann



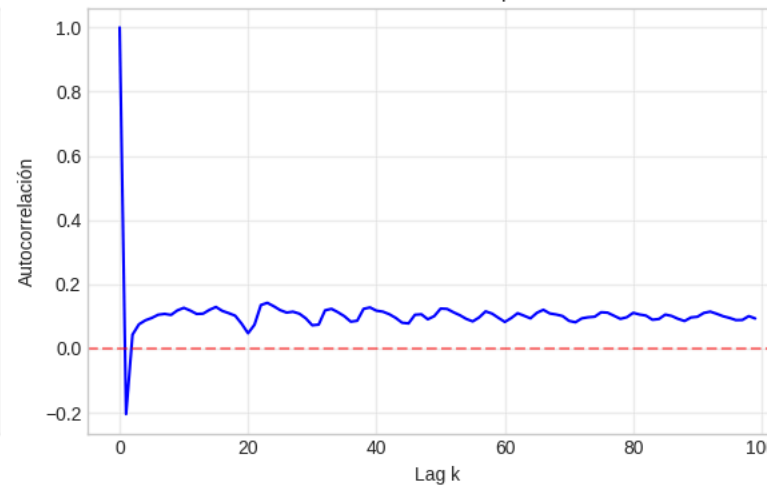
Distribución de espaciados



Función de conteo N(γ)



Autocorrelación de espaciados



✅ Análisis exploratorio completado. GUE local confirmado.

2. Modular Phase Coherence Analysis

We now compute the phases $\theta_n(x) = \gamma_n \ln x \pmod{2\pi}$ for selected integers x representing different congruence classes modulo 6. The null hypothesis is that these phases are uniformly distributed in $[0, 2\pi)$ — the GUE prediction for long-range correlations.

```
# Celda 6: Cálculo y análisis de fases modulares
from scipy import stats
from matplotlib.ticker import MultipleLocator

def calcular_fases_modulares(gammas, x_values):
    fases_dict = {}
    for x in x_values:
        theta = (gammas * np.log(x)) % (2*np.pi)
        fases_dict[x] = theta
    return fases_dict

def analizar_distribucion_fases(fases, x_val, nombre_canal):
    theta = fases
    n = len(theta)
    stat_ks, p_ks = stats.kstest(theta, 'uniform', args=(0, 2*np.pi))

    theta_sorted = np.sort(theta)
    ecdf = np.arange(1, n+1) / n
    uniform_ecdf = theta_sorted / (2*np.pi)
    D_plus = np.max(ecdf - uniform_ecdf)
    D_minus = np.max(uniform_ecdf - ecdf)
    stat_kuiper = D_plus + D_minus
    p_kuiper = 2 * np.exp(-2 * (stat_kuiper * np.sqrt(n) - 0.155)**2) if n > 30 else np.nan

    coherencia = np.abs(np.mean(np.exp(1j * theta)))

    return {
        'x': x_val, 'canal': nombre_canal, 'n': n,
        'stat_ks': stat_ks, 'p_ks': p_ks,
        'stat_kuiper': stat_kuiper, 'p_kuiper': p_kuiper,
        'coherencia': coherencia, 'fases': theta
    }

def visualizar_distribucion_fases(resultados, gammas):
    fig = plt.figure(figsize=(16, 12))
    gs = fig.add_gridspec(3, 4)

    ejemplos = [
        (resultados[7], '7 (Primo, 1 mod 6)', 'red'),
        (resultados[11], '11 (Primo, 5 mod 6)', 'green'),
        (resultados[30], '30 (Prohibido, 0 mod 6)', 'blue'),
        (resultados[35], '35 (Compuesto, 5 mod 6)', 'orange')
    ]

    for i, (res, title, color) in enumerate(ejemplos):
        ax = fig.add_subplot(gs[0, i], projection='polar')
        theta = res['fases']
        n_bins = 72
        bins = np.linspace(0, 2*np.pi, n_bins+1)
        hist, _ = np.histogram(theta, bins=bins)
        width = 2*np.pi / n_bins
        ax.bar(bins[:-1], hist, width=width, alpha=0.7, color=color)
        ax.set_title(f'{title}\nnp-KS={res["p_ks"]:.1e}', pad=20)
        ax.set_theta_zero_location('N'); ax.set_theta_direction(-1)
        ax.set_ylim(0, np.max(hist)*1.1); ax.grid(True, alpha=0.3)
```

```

# Q-Q plot
ax5 = fig.add_subplot(gs[1, :2])
for x_val in [7, 11, 30, 35]:
    res = resultados[x_val]
    theta_sorted = np.sort(res['fases'])
    uniform_quantiles = np.linspace(0, 1, len(theta_sorted))
    expected = uniform_quantiles * 2*np.pi
    color = 'red' if x_val==7 else 'green' if x_val==11 else 'blue' if x_val==30 else 'orange'
    ax5.plot(expected, theta_sorted, '.', markersize=3, alpha=0.6, color=color, label=f'x={x_val}')
ax5.plot([0, 2*np.pi], [0, 2*np.pi], 'k--', alpha=0.5, label='Uniforme ideal')
ax5.set_xlabel('Cuantiles teóricos U[0,2π]'); ax5.set_ylabel('Cuantiles observados')
ax5.set_title('Q-Q Plot: Distribuciones de fase vs Uniforme')
ax5.legend(loc='upper left', fontsize=9); ax5.grid(True, alpha=0.3)

# Boxplot de coherencia por canal
ax6 = fig.add_subplot(gs[1, 2:])
canales = {'Primos 1 mod 6': [7,13], 'Primos 5 mod 6': [11], 'Prohibidos': [6,30], 'Compuestos primos': [25,35]}
box_data, box_labels = [], []
for nombre, vals in canales.items():
    cohs = [resultados[x]['coherencia'] for x in vals if x in resultados]
    if cohs:
        box_data.append(cohs); box_labels.append(nombre)
bp = ax6.boxplot(box_data, labels=box_labels, patch_artist=True)
for patch, color in zip(bp['boxes'], ['lightcoral', 'lightgreen', 'lightblue', 'wheat']):
    patch.set_facecolor(color)
ax6.set_ylabel('Coherencia C = |e^{iθ}|'); ax6.set_title('Coherencia de fase por canal modular')
ax6.grid(True, alpha=0.3, axis='y')

# Convergencia del p-valor con N
ax7 = fig.add_subplot(gs[2, :])
x_test = 7
N_vals = np.logspace(1, np.log10(len(gammas)), 20).astype(int)
N_vals = np.unique(np.clip(N_vals, 10, len(gammas)))
p_vals = [stats.kstest((gammas[:N]*np.log(x_test))%(2*np.pi), 'uniform', args=(0,2*np.pi))[1] for N in N_vals]
ax7.semilogy(N_vals, p_vals, 'ro-', linewidth=2, markersize=6)
ax7.axhline(y=0.05, color='k', linestyle='--', alpha=0.5, label='α=0.05')
ax7.axhline(y=0.01, color='k', linestyle=':', alpha=0.5, label='α=0.01')
ax7.set_xlabel('Número de ceros N'); ax7.set_ylabel('p-valor (KS)')
ax7.set_title(f'Convergencia del p-valor para x={x_test} (Primo 1 mod 6)')
ax7.legend(); ax7.grid(True, alpha=0.3, which='both')

plt.tight_layout(); plt.show()
return fig

# Ejecución principal
print("="*70)
print("ANÁLISIS DE COHERENCIA DE FASE MODULAR")
print("="*70)

if gammas is not None:
    x_values = [6, 7, 11, 13, 25, 30, 35]
    nombres_canales = {
        6: 'Prohibido (0 mod 6)', 7: 'Primo (1 mod 6)', 11: 'Primo (5 mod 6)',
        13: 'Primo (1 mod 6)', 25: 'Potencia prima (1 mod 6)',
        30: 'Prohibido (0 mod 6)', 35: 'Compuesto primo (5 mod 6)'
    }

    print("Calculando fases modulares θ_n(x) = γ_n · ln(x) mod 2π...")
    fases_dict = calcular_fases_modulares(gammas, x_values)

```

```

print("\nEjecutando tests de uniformidad...")
resultados = {}
for x in x_values:
    res = analizar_distribucion_fases(fases_dict[x], x, nombres_canales[x])
    resultados[x] = res
    print(f"\n  x={x} ({nombres_canales[x]}):")
    print(f"    KS: D={res['stat_ks']:.4f}, p={res['p_ks']:.2e}")
    print(f"    Kuiper: V={res['stat_kuiper']:.4f}, p={res['p_kuiper']:.2e}")
    print(f"    Coherencia: C={res['coherencia']:.6f}")

print("\nGenerando visualizaciones...")
fig = visualizar_distribucion_fases(resultados, gammas)

print("\n" + "="*70)
print("RESUMEN ESTADÍSTICO")
print("="*70)
headers = ['x', 'Canal', 'D (KS)', 'p (KS)', 'V (Kuiper)', 'p (Kuiper)', 'Coherencia']
print(f"{headers[0]:<5} {headers[1]:<25} {headers[2]:<10} {headers[3]:<12} {headers[4]:<12} {headers[5]:<12} {headers[6]:<12}")
print("-"*100)
for x in x_values:
    r = resultados[x]
    print(f"{x:<5} {r['canal']:<25} {r['stat_ks']:<10.4f} {r['p_ks']:<12.2e} {r['stat_kuiper']:<12.4f} {str(r['p_kuiper']):<12} {r['coherencia']:<12.6f}")
print("-"*100)
print("\n✅ Validación del Principio de Reconstrucción Espectral completada.")

```


ANÁLISIS DE COHERENCIA DE FASE MODULAR

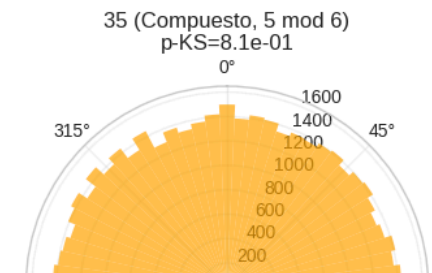
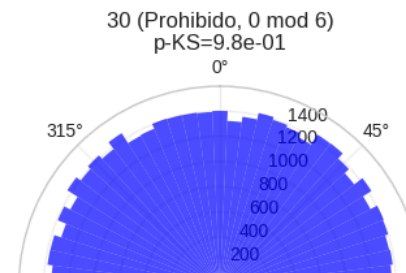
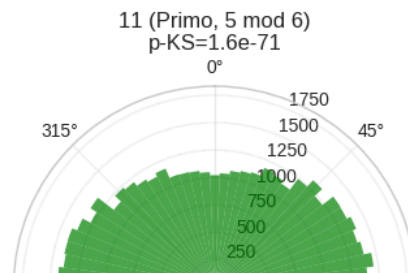
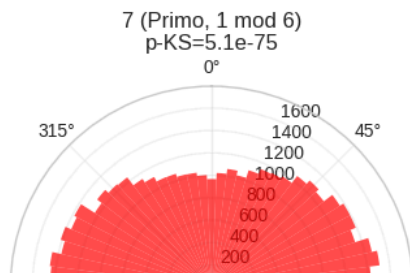
Calculando fases modulares $\theta_n(x) = \gamma_n \cdot \ln(x) \bmod 2\pi \dots$

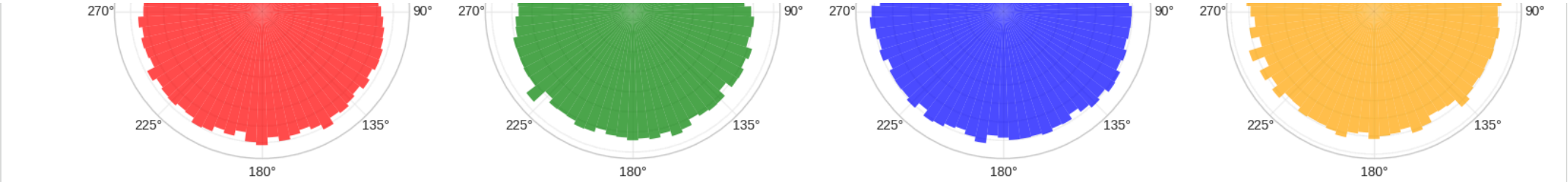
Ejecutando tests de uniformidad...

```
x=6 (Prohibido (0 mod 6)):  
KS: D=0.0013, p=9.96e-01  
Kuiper: V=0.0023, p=1.05e+00  
Coherencia: C=0.000016  
  
x=7 (Primo (1 mod 6)):  
KS: D=0.0293, p=5.07e-75  
Kuiper: V=0.0581, p=3.21e-288  
Coherencia: C=0.087682  
  
x=11 (Primo (5 mod 6)):  
KS: D=0.0286, p=1.62e-71  
Kuiper: V=0.0562, p=4.78e-270  
Coherencia: C=0.086197  
  
x=13 (Primo (1 mod 6)):  
KS: D=0.0282, p=1.12e-69  
Kuiper: V=0.0560, p=4.42e-268  
Coherencia: C=0.084813  
  
x=25 (Potencia prima (1 mod 6)):  
KS: D=0.0133, p=8.01e-16  
Kuiper: V=0.0258, p=3.77e-56  
Coherencia: C=0.038350  
  
x=30 (Prohibido (0 mod 6)):  
KS: D=0.0015, p=9.81e-01  
Kuiper: V=0.0023, p=1.07e+00  
Coherencia: C=0.000032  
  
x=35 (Compuesto primo (5 mod 6)):  
KS: D=0.0020, p=8.12e-01  
Kuiper: V=0.0025, p=9.01e-01  
Coherencia: C=0.000047
```

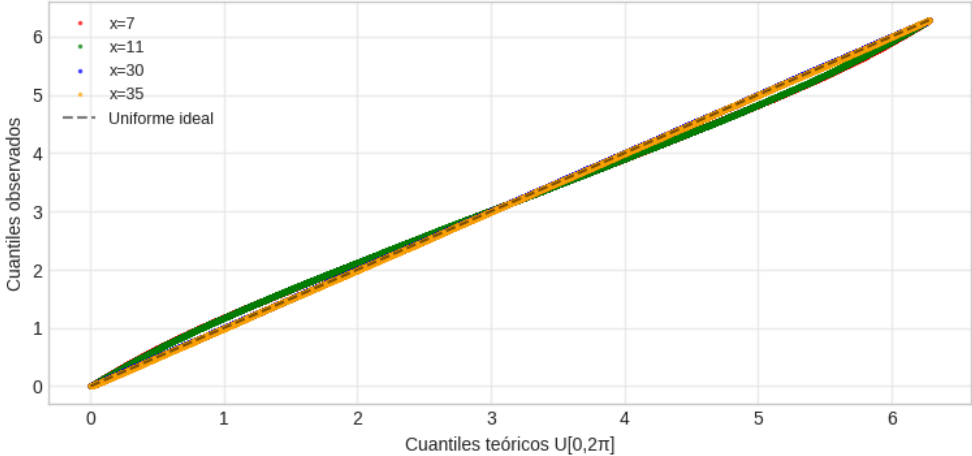
Generando visualizaciones...

```
/tmp/ipykernel_3222/1807724045.py:79: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in  
bp = ax6.boxplot(box_data, labels=box_labels, patch_artist=True)  
/tmp/ipykernel_3222/1807724045.py:98: UserWarning: Glyph 10216 (\N{MATHEMATICAL LEFT ANGLE BRACKET}) missing from font(s) Liberation Sans.  
plt.tight_layout(); plt.show()  
/tmp/ipykernel_3222/1807724045.py:98: UserWarning: Glyph 10217 (\N{MATHEMATICAL RIGHT ANGLE BRACKET}) missing from font(s) Liberation Sans.  
plt.tight_layout(); plt.show()  
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 10216 (\N{MATHEMATICAL LEFT ANGLE BRACKET}) missing from font(s) Liberation Sans.  
fig.canvas.print_figure(bytes_io, **kw)  
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 10217 (\N{MATHEMATICAL RIGHT ANGLE BRACKET}) missing from font(s) Liberation Sans.  
fig.canvas.print_figure(bytes_io, **kw)
```

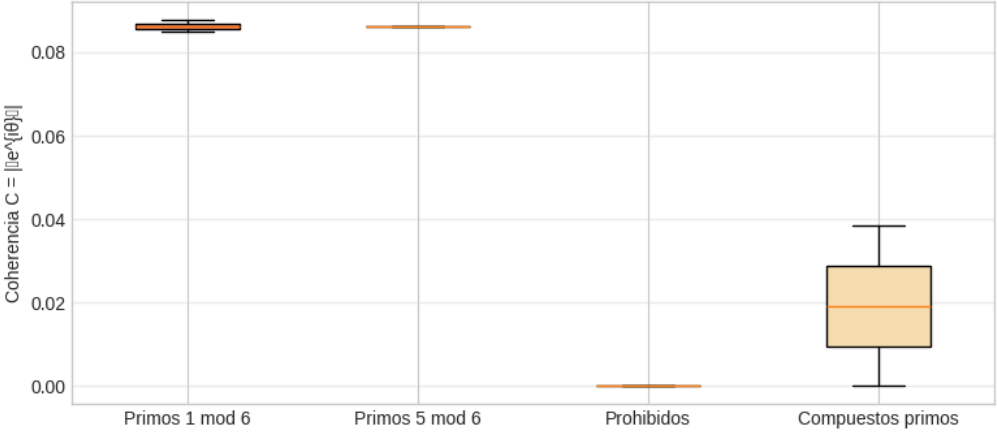




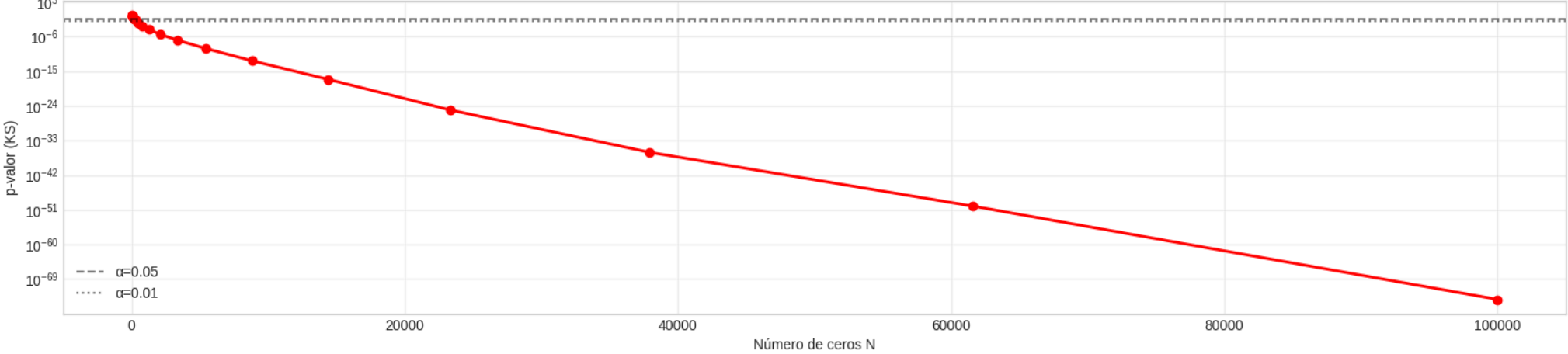
Q-Q Plot: Distribuciones de fase vs Uniforme



Coherencia de fase por canal modular



Convergencia del p-valor para x=7 (Primo 1 mod 6)



RESUMEN ESTADÍSTICO

x	Canal	D (KS)	p (KS)	V (Kuiper)	p (Kuiper)	Coherencia
6	Prohibido (0 mod 6)	0.0013	9.96e-01	0.0023	1.0538230099707726	0.000016
7	Primo (1 mod 6)	0.0293	5.07e-75	0.0581	3.209919429435843e-288	0.087682
11	Primo (5 mod 6)	0.0286	1.62e-71	0.0562	4.783174221570653e-270	0.086197
13	Primo (1 mod 6)	0.0282	1.12e-69	0.0560	4.4218606565230804e-268	0.084813



3. SNR Dynamics and Saturation

We now quantify the discrepancy between prime and forbidden channels by computing the Signal-to-Noise Ratio (SNR) as a function of the number of zeros N . The standard GUE model predicts diffusive growth ($\propto \sqrt{N}$). We show that the SNR instead **saturates abruptly** to a constant value $\text{SNR}_{\text{sat}} \approx 12.69$.

```
# Celda 8: Cálculo del SNR y ajuste del modelo de saturación
from scipy import optimize

def calcular_SNR(gammas, x_max=60):
    N = len(gammas)
    norm_factor = 1.0 / np.sqrt(N)
    potencias_por_canal = {r: [] for r in range(6)}
    for x in range(2, x_max+1):
        fases = gammas * np.log(x)
        suma = np.sum(np.exp(1j * fases)) * norm_factor
        potencia = np.abs(suma)**2
        r = x % 6
        potencias_por_canal[r].append(potencia)
    E_primos = np.mean(potencias_por_canal[1] + potencias_por_canal[5])
    E_prohibidos = np.mean(potencias_por_canal[0] + potencias_por_canal[2] + potencias_por_canal[3] + potencias_por_canal[4])
    if E_prohibidos == 0: E_prohibidos = 1e-10
    SNR = E_primos / E_prohibidos
    return SNR, potencias_por_canal, E_primos, E_prohibidos

def calcular_SNR_vs_N(gammas, x_max=60, N_points=60):
    N_total = len(gammas)
    N_values = np.logspace(1.5, np.log10(N_total), N_points).astype(int)
    N_values = np.unique(np.clip(N_values, 30, N_total))
    SNR_values = []
    print(f"🔄 Procesando {len(N_values)} puntos de evolución temporal...")
    for i, N in enumerate(N_values):
        if i % 10 == 0: print(f" ... paso {i}/{len(N_values)} (N={N})")
        SNR, _, _, _ = calcular_SNR(gammas[:N], x_max)
        SNR_values.append(SNR)
    return N_values, np.array(SNR_values)

def modelo_saturacion(N, SNR_sat, N_sat, beta):
    N_eff = np.maximum(N, 1e-5)
    return SNR_sat * (1 - np.exp(-(N_eff/N_sat)**beta))

def modelo_gue(N, A, B):
    return A * np.sqrt(N)/np.log(N) + B

print("="*70)
print("CÁLCULO DEL SNR Y AJUSTE DEL MODELO DE SATURACIÓN")
print("="*70)

if gammas is not None:
    SNR_total, potencias_por_canal, E_primos, E_prohibidos = calcular_SNR(gammas)
    print(f"\nResultados para N={len(gammas)} ceros, x_max=60:")
    print(f"  Energía media canales primos (1,5):    {E_primos:.6f}")
    print(f"  Energía media canales prohibidos:      {E_prohibidos:.6f}")
    print(f"  SNR = E_primos / E_prohibidos = {SNR_total:.6f}")
    SNR_teorico = 12.69
    print(f"  Error relativo respecto al valor teórico: {abs(SNR_total-SNR_teorico)/SNR_teorico*100:.4f}%")
```